



# KryptoX

Crypto Market Intelligence App

## COMPLETE PROJECT DOCUMENTATION

Architecture · Dart backend · REST API · Flutter app  
Screens & UI · Business logic · Design system · Animations  
Setup, testing, verification status and production roadmap

### Flutter + Dart

100% Dart stack

### 4 screens

+ splash & shell

### 8 REST endpoints

shelf backend

### ~7,250 lines

of source code

# Contents

---

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Project Overview</b>                                      | <b>4</b>  |
| 1.1 What the app does                                        | 4         |
| 1.2 Requirements and how each is met                         | 4         |
| 1.3 Technology summary                                       | 5         |
| <b>System Architecture</b>                                   | <b>6</b>  |
| 2.1 Why use a backend instead of calling CoinGecko directly? | 6         |
| 2.2 Repository structure                                     | 7         |
| <b>Backend (Dart + shelf)</b>                                | <b>8</b>  |
| 3.1 Running the server                                       | 8         |
| 3.2 How a request is handled                                 | 8         |
| 3.3 Data layer: CoinGecko client and cache                   | 8         |
| 3.4 Demo data (lib/mock_data.dart)                           | 10        |
| 3.5 REST API reference                                       | 10        |
| 3.6 Backend verification                                     | 11        |
| <b>Flutter App Architecture</b>                              | <b>12</b> |
| 4.1 Layers                                                   | 12        |
| 4.2 State management                                         | 12        |
| 4.3 Networking and error handling                            | 13        |
| 4.4 When the app calls the backend                           | 13        |
| <b>Design System &amp; Motion</b>                            | <b>14</b> |
| 5.1 Colour tokens (KxColors)                                 | 14        |
| 5.2 Typography (google_fonts)                                | 14        |
| 5.3 Component library (lib/widgets)                          | 14        |
| 5.4 Motion catalogue                                         | 15        |
| <b>Screens: UI and Logic</b>                                 | <b>17</b> |
| 6.1 Splash screen (splash_screen.dart)                       | 17        |
| 6.2 App shell and navigation (home_shell.dart)               | 17        |
| 6.3 Markets screen (coin_list_screen.dart)                   | 18        |
| 6.4 Coin details screen (coin_detail_screen.dart)            | 19        |
| 6.5 Market stats screen (market_stats_screen.dart)           | 20        |
| 6.6 Watchlist screen (watchlist_screen.dart)                 | 21        |
| <b>Performance, Quality &amp; Robustness</b>                 | <b>22</b> |

|                                                      |           |
|------------------------------------------------------|-----------|
| 7.1 Performance decisions . . . . .                  | 22        |
| 7.2 Robustness . . . . .                             | 22        |
| 7.3 Unit tests (test/formatters_test.dart) . . . . . | 22        |
| <b>Setup, Run &amp; Test Guide . . . . .</b>         | <b>23</b> |
| 8.1 Requirements . . . . .                           | 23        |
| 8.2 Steps . . . . .                                  | 23        |
| 8.3 Testing the backend by hand . . . . .            | 23        |
| 8.4 Troubleshooting . . . . .                        | 23        |
| <b>Verification Status &amp; Roadmap . . . . .</b>   | <b>24</b> |
| 9.1 Verification status . . . . .                    | 24        |
| 9.2 Dependency note . . . . .                        | 24        |
| 9.3 Limitations and production roadmap . . . . .     | 24        |
| <b>Appendix: Source File Inventory . . . . .</b>     | <b>25</b> |

## SECTION 01

# Project Overview

KryptoX is a CoinGecko-style crypto market research app. A Flutter mobile app shows live market data from a small Dart backend, which fetches it from the public CoinGecko API, caches it, and falls back to demo data when the API is unavailable.

## 1.1 What the app does

- Lists the top 100 cryptocurrencies with live price, 24h change, market cap, volume and a 7-day sparkline.
- Lets users search, filter (Top 10, Gainers, Losers, Stablecoins) and sort (market cap, price, volume, 24h/7d change, name, ascending or descending).
- Shows a research page per coin: an interactive price chart (24H to 1Y), a 24h range bar, performance figures, market cap, volume, fully diluted valuation, all-time high/low, supply and a description.
- Shows a global market dashboard: total market cap and volume, BTC/ETH dominance, an animated dominance donut chart, an estimated sentiment gauge and top movers.
- Keeps a watchlist that is saved on the backend, with a summary card, sorting and swipe-to-remove with undo.
- Handles loading, error and empty states on every screen, and labels every screen's data as LIVE, CACHED or DEMO.

## 1.2 Requirements and how each is met

| Requirement (task brief)                                                             | How KryptoX meets it                                                                                                                                                                       | Where                                                           |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Flutter crypto market/research mobile app</b>                                     | Flutter 3.38+ app, dark design with glass panels and neon accents, Material 3                                                                                                              | flutter_app/                                                    |
| <b>Screens: coin list, coin details, market statistics, watchlist</b>                | Markets, Coin details, Stats and Watchlist screens, plus an animated splash screen and the app shell with a floating nav bar                                                               | lib/screens/                                                    |
| <b>Integrate a crypto REST API; show live/mock price, volume, market cap, supply</b> | The backend proxies CoinGecko and falls back to built-in demo data. Price, volume, market cap and circulating/total/max supply are shown in the list, the details page and the stats page. | backend/,<br>api_service.dart                                   |
| <b>Search, filtering, sorting, watchlist</b>                                         | Debounced search, 5 filters and 6 sort fields handled on the server, with sort direction; the watchlist is stored on the server and has its own client-side sort                           | server.dart,<br>coin_list_screen.dart,<br>watchlist_screen.dart |
| <b>Interactive price chart</b>                                                       | fl_chart line chart with 5 ranges, touch-and-drag scrubbing, a tooltip, and the header price following the finger                                                                          | price_chart.dart,<br>coin_detail_screen.dart                    |
| <b>API loading, error and empty states</b>                                           | Shimmer skeletons, a full-screen error with Retry, an error banner when a refresh fails with data still shown, empty states with actions, and inline errors for partial failures           | state_views.dart,<br>Loadable                                   |
| <b>Connect Flutter to a simple backend</b>                                           | Dart shelf server with 8 REST endpoints, caching, fallback and a persisted watchlist                                                                                                       | backend/bin/server.da<br>rt                                     |
| <b>Working mini CoinGecko-style app</b>                                              | End-to-end: app → backend → CoinGecko, including an offline demo mode                                                                                                                      | whole project                                                   |

## 1.3 Technology summary

| Layer       | Technology                                                | Packages (all from pub.dev)                                                               |
|-------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Mobile app  | Flutter 3.38+, Dart 3.10+                                 | provider, http, fl_chart, flutter_animate, animations, google_fonts, cached_network_image |
| Backend     | Dart 3 console server                                     | shelf, shelf_router, http                                                                 |
| Data source | CoinGecko public REST API v3                              | No key needed; an optional free demo key raises rate limits                               |
| Storage     | watchlist.json on disk; in-memory cache                   | None (dart:io)                                                                            |
| Tests       | flutter_test unit tests; backend endpoint tests with curl | flutter_test, flutter_lints                                                               |

## SECTION 02

# System Architecture

The app never calls CoinGecko itself. Every request goes through the KryptoX backend, which gives the app one stable data format, protects against rate limits and keeps the app working offline.

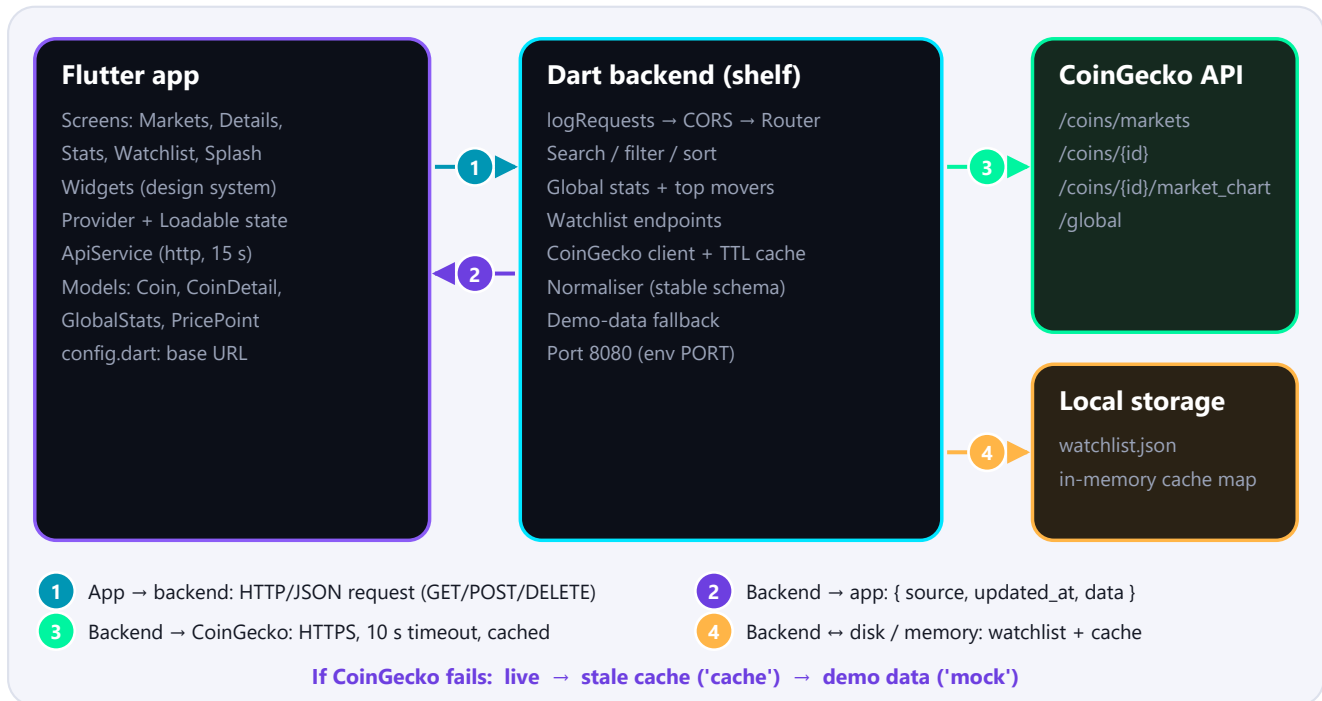


Figure 1: Components and data flow.

## 2.1 Why use a backend instead of calling CoinGecko directly?

| Benefit                      | Explanation                                                                                                                                                                          |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stable data format</b>    | CoinGecko's JSON is large and deeply nested. The backend reshapes it into a small, flat format that the Flutter models depend on, so a change at CoinGecko only needs a backend fix. |
| <b>Rate-limit protection</b> | The free API allows about 30 calls a minute. Responses are cached for 60 s to 5 min, so any number of app users or refreshes cost at most about one upstream list call per minute.   |
| <b>Resilience</b>            | If CoinGecko fails, the backend serves stale cached data, and if nothing is cached it serves demo data. The app never shows a blank screen because of an upstream problem.           |
| <b>Server-side logic</b>     | Search, filtering, sorting, global statistics, top movers and the watchlist are all handled on the server.                                                                           |
| <b>Browser support</b>       | The backend sends CORS headers, so the same app can also run in Chrome (Flutter web).                                                                                                |

## 2.2 Repository structure

```

newidea/
├─ README.md                quick start + API reference
├─ KryptoX_Project_Documentation.pdf  this document
├─ backend/                 Dart REST backend
│   └─ pubspec.yaml         kryptox_backend: shelf, shelf_router, http
│   └─ bin/server.dart       routes, filter/sort, global stats, watchlist, CORS  (235 lines)
│   └─ lib/coingecko.dart    upstream client, TTL cache, normalisation  (107 lines)
│   └─ lib/mock_data.dart    deterministic offline demo data  (153 lines)
├─ flutter_app/            Flutter client (package: kryptox)
│   └─ pubspec.yaml         dependencies, Flutter >= 3.38
│   └─ setup.ps1            generates platform folders + Android HTTP permissions
│   └─ test/formatters_test.dart  unit tests
│   └─ lib/
│       └─ main.dart         providers, theme, system UI, splash as home
│       └─ config.dart       backend base URL (emulator / localhost / --dart-define)
│       └─ models/coin.dart  Coin, CoinDetail, PricePoint, GlobalStats (+ JSON parsing)
│       └─ services/api_service.dart  typed REST client, errors, CoinSort / CoinFilter enums
│       └─ state/loadable.dart  generic loading/error/data holder
│       └─ state/watchlist_provider.dart  optimistic watchlist state
│       └─ theme/app_theme.dart  design tokens: colours, fonts, page transitions
│       └─ widgets/          design system (glass, background, price, chart, controls...)
│       └─ screens/          splash, shell, markets, detail, stats, watchlist

```

## SECTION 03

## Backend (Dart + shelf)

A small Dart HTTP server. It is the app's only data source: it fetches and caches CoinGecko data, reshapes it, runs search, filter and sort, builds market statistics and stores the watchlist.

### 3.1 Running the server

```
cd backend
dart pub get          # download packages (first time only)
dart run bin/server.dart # → KryptoX backend on http://localhost:8080 (mock only: false)

# demo mode (no internet needed):
$env:USE MOCK="1"; dart run bin/server.dart
```

| Environment variable | Default | Effect                                                                                                                    |
|----------------------|---------|---------------------------------------------------------------------------------------------------------------------------|
| PORT                 | 8080    | Port to listen on. The server listens on all network interfaces (0.0.0.0), so a real phone on the same Wi-Fi can connect. |
| USE MOCK=1           | off     | Always serve demo data and never call CoinGecko. Useful for demos, offline development and exams.                         |
| COINGECKO_API_KEY    | none    | Sent as the header x-cg-demo-api-key. A free CoinGecko demo key gives higher rate limits.                                 |

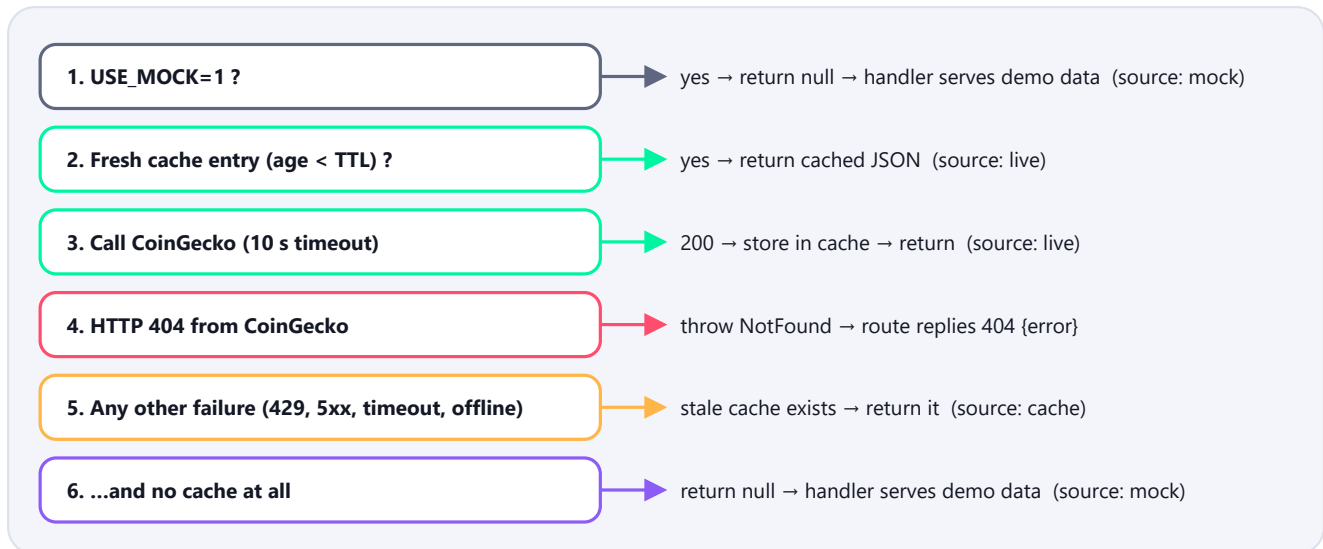
### 3.2 How a request is handled

Every request passes through a shelf **Pipeline** before it reaches a route handler:

```
HTTP request
|
▼ logRequests()    prints time, method, path, status and duration to the console
▼ _cors()         answers OPTIONS preflight requests; adds Access-Control-Allow-* headers to every response
▼ Router          matches method + path; <id> segments become handler arguments
▼ handler         e.g. _listCoins(Request) or _coinDetail(Request, String id)
|
▼
JSON response { "source": "...", "updated_at": <unix seconds>, "data": ... } or { "error": "..." }
```

### 3.3 Data layer: CoinGecko client and cache

All upstream calls go through one method, `CoinGecko.get(path, params, ttl)` in `lib/coingecko.dart`. The full request URL is the cache key, and the cache is an in-memory `Map<String, (DateTime, dynamic)>`. The method works like this:

Figure 2: What `CoinGecko.get` does, step by step.

| Upstream endpoint                                  | Used by                                 | Cache time | Most calls per minute   |
|----------------------------------------------------|-----------------------------------------|------------|-------------------------|
| /coins/markets (top 100, sparkline, 24h+7d change) | /api/coins, /api/global, /api/watchlist | 60 s       | 1 (shared by all three) |
| /global                                            | /api/global                             | 2 min      | 0.5                     |
| /coins/{id}                                        | /api/coins/{id}                         | 2 min      | 0.5 per coin            |
| /coins/{id}/market_chart?days=N                    | /api/coins/{id}/chart                   | 5 min      | 0.2 per coin per range  |

### Where the data came from (the 'source' field)

| Value | Meaning                                                                                          | Badge in the app        |
|-------|--------------------------------------------------------------------------------------------------|-------------------------|
| live  | Fresh from CoinGecko, or from a cache entry still within its cache time                          | Green pulsing dot: LIVE |
| cache | CoinGecko failed, so this is older cached data                                                   | Amber: CACHED           |
| mock  | CoinGecko failed and nothing was cached, or <code>USE MOCK=1</code> ; this is built-in demo data | Violet: DEMO            |

### Reshaping CoinGecko's data

- `normalizeMarket()` flattens a `/coins/markets` item into KryptoX's list format, preferring `price_change_percentage_24h_in_currency` over the plain field, and turning the nested sparkline into a flat list of prices.
- `normalizeDetail()` flattens `market_data.*.usd` fields. It keeps only the first paragraph of the description, strips HTML tags, cuts it to 1,200 characters, keeps the first non-empty homepage and at most 5 categories.
- As a result, the Flutter models never see CoinGecko's own field names.

### 3.4 Demo data (lib/mock\_data.dart)

- 20 real coins (BTC, ETH, USDT, BNB, SOL, USDC, XRP, DOGE, ADA, TRX, AVAX, LINK, DOT, LTC, SHIB, UNI, NEAR, APT, XLM, XMR) with realistic price, market cap, volume, 24h/7d change and supply numbers.
- Charts are generated as a random walk, scaled so that the last point equals the coin's current price.
- Charts are the same on every run: each chart's random seed comes from a fixed FNV-1a hash of a key such as "solana-30".
- Chart resolution: 288 points for 1 day (every 5 min); hourly up to 30 days (at most 400 points); daily beyond that. Volatility grows with the range (0.3%, 0.8%, 2.5% per step).
- Details (all-time high/low, fully diluted valuation, 30d/1y change) and global stats (total market cap, dominance) are calculated from the same coin list, so all the numbers agree with each other.

### 3.5 REST API reference

Every successful data response uses the same wrapper. Errors return **400** (bad input) or **404** (unknown coin) with an error message.

```
{ "source": "live" | "cache" | "mock", "updated_at": 1790000000, "data": <payload> }
{ "error": "Invalid sort: bogus" }
```

| Method        | Path                        | Purpose                                              |
|---------------|-----------------------------|------------------------------------------------------|
| GET           | /api/health                 | Health check → {status: ok, mock_mode}               |
| GET           | /api/coins                  | Coin list with search / filter / sort                |
| GET           | /api/coins/{id}             | Full research data for one coin                      |
| GET           | /api/coins/{id}/chart?days= | Price history as [[timestamp_ms, price], ...]        |
| GET           | /api/global                 | Global market stats + top gainers, losers and volume |
| GET           | /api/watchlist              | Full coin data for the watchlist                     |
| GET           | /api/watchlist/ids          | Watchlist ids only                                   |
| POST / DELETE | /api/watchlist/{id}         | Add / remove a coin; returns the new id list         |

#### GET /api/coins: search, filter and sort

| Parameter | Values                                                      | Server logic                                                                                                                               |
|-----------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| search    | any text                                                    | Case-insensitive substring match on name OR symbol ('bit' finds Bitcoin, Bitcoin Cash, Bittensor, ...).                                    |
| filter    | all · top10 · gainers · losers · stablecoins                | gainers: 24h change > 0 · losers: < 0 · top10: rank ≤ 10 · stablecoins: price between \$0.97 and \$1.03 with less than 1% movement in 24h. |
| sort      | market_cap · price · volume · change_24h · change_7d · name | Any other value → 400. 'name' sorts alphabetically, ignoring case.                                                                         |
| order     | desc (default) · asc                                        | Coins with no value for the sort field always go to the bottom, in either direction.                                                       |

The handler works in five steps: get the top 100 (from the cache when possible) → search → filter → sort → wrap.  
Example:

```
GET /api/coins?search=bit&sort=price&order=desc
→ { "source": "live", "updated_at": 1790000000, "data": [
  { "id": "bitcoin", "symbol": "btc", "name": "Bitcoin", "image": "https://...",
    "current_price": 84034, "market_cap": 1.66e12, "market_cap_rank": 1, "total_volume": 3.1e10,
    "high_24h": 84800, "low_24h": 82950, "price_change_percentage_24h": 1.84,
    "price_change_percentage_7d": 4.12, "circulating_supply": 19740000,
    "total_supply": 21000000, "max_supply": 21000000, "sparkline": [ ...168 hourly prices... ] },
  ... ] }
```

### GET /api/coins/{id}

Calls CoinGecko /coins/{id} with localization, tickers, community data and developer data all turned off, which keeps the response small. The response has every list field plus: price\_change\_percentage\_30d / \_1y, fully\_diluted\_valuation, ath, ath\_change\_percentage, ath\_date, atl, atl\_date, description, homepage, categories and genesis\_date. An unknown id returns 404.

### GET /api/coins/{id}/chart?days=1|7|30|90|365

Returns {days, prices: [[timestamp\_ms, price], ...]}. Any other days value returns 400.

### GET /api/global

Combines two sources: (1) CoinGecko /global: total market cap, total volume, 24h market cap change, number of active coins, number of markets and dominance percentages; (2) the cached coin list, from which the server builds top\_gainers (5), top\_losers (5) and top\_volume (5). If either source is demo data, the whole response is marked mock, so the badge is never wrong.

### Watchlist endpoints

- Stored as a JSON array of coin ids in `watchlist.json`, in the folder the server was started from.
- POST adds the id if it isn't already there (adding the same coin twice has no effect); DELETE removes it. Both return {ids: [...]}.
- GET /api/watchlist returns full coin data from the cached top-100 list, in the order the coins were added.
- Synchronous file reads and writes are safe here because a Dart server handles one request at a time on a single thread. A missing or corrupted file is treated as an empty list.

## 3.6 Backend verification

The backend was run and tested with the Dart 3.13 SDK. **dart analyze** reported no issues. These live checks against CoinGecko passed:

| Check                                                             | Result                                                                                      |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| GET /api/health                                                   | {status: ok, mock_mode: false}                                                              |
| Search 'bit', sort by price desc                                  | live, 7 coins: Bitcoin 84,034 · Bitcoin Cash 337.11 · Bittensor 331.36 · WhiteBIT 83.88 ... |
| Filter gainers, sort by 24h change                                | 72 coins; top: Quant +25.86%                                                                |
| Details for ethereum                                              | live, price 2,687.73, all-time high 4,946.05, description returned                          |
| 30-day bitcoin chart                                              | live, 721 points                                                                            |
| Global stats                                                      | live, total market cap 2.89 T, 5 top gainers                                                |
| Add, list and remove solana on the watchlist                      | {ids:[solana]} → Solana → {ids:[]}                                                          |
| Unknown coin / sort=bogus                                         | 404 / 400                                                                                   |
| Demo mode: stablecoins filter, details, 1-day chart, global stats | Tether and USDC; details OK; 288 points ending at the current price; top losers OK          |

## SECTION 04

# Flutter App Architecture

The app is split into layers: models (data) → service (network) → state (loading, error and data) → widgets (design system) → screens. Provider supplies shared objects, and each screen keeps its own loading state in Loadable objects.

## 4.1 Layers

| Layer          | Files                         | Responsibility                                                                                                                                                                                                  |
|----------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Config</b>  | config.dart                   | Backend base URL: 10.0.2.2:8080 on the Android emulator, localhost:8080 elsewhere, or --dart-define=API_BASE_URL=...                                                                                            |
| <b>Models</b>  | models/coin.dart              | Coin, CoinDetail, PricePoint and GlobalStats. Parsing tolerates ints where decimals are expected, nulls and missing fields; null items inside lists are skipped.                                                |
| <b>Service</b> | services/api_service.dart     | Typed methods for every endpoint; builds URLs and query strings; 15 s timeout; turns every failure into an ApiException with a readable message; defines the CoinSort and CoinFilter enums with display labels. |
| <b>State</b>   | state/loadable.dart           | Loadable<T> (a ChangeNotifier) holds data, source, updatedAt, error and loading. It keeps the old data while reloading, and ignores a response if a newer request has already been sent.                        |
| <b>State</b>   | state/watchlist_provider.dart | Shared set of watchlist ids with optimistic toggling and rollback on failure.                                                                                                                                   |
| <b>Theme</b>   | theme/app_theme.dart          | Design tokens (KxColors, KxText), Material 3 ThemeData, page transitions.                                                                                                                                       |
| <b>Widgets</b> | widgets/*.dart                | Reusable design system: background, glass cards, animated price, chart, sparkline, controls, nav bar, state views.                                                                                              |
| <b>Screens</b> | screens/*.dart                | Splash, HomeShell, CoinListScreen, CoinDetailScreen, MarketStatsScreen, WatchlistScreen, and the openCoin() navigation helper.                                                                                  |

## 4.2 State management

```
main()
├─ MultiProvider
│  ├─ Provider<ApiService>           one shared HTTP client for the whole app
│  └─ ChangeNotifierProvider<WatchlistProvider>  ..load() on start → which stars are filled
│      └─ MaterialApp(theme: buildKxTheme(), home: SplashScreen)
└─ Each screen:  final _coins = Loadable<List<Coin>>();
                  _coins.load(() => context.read<ApiService>().getCoins(...));
                  ListenableBuilder(listenable: _coins, builder: ...) // rebuilds only that part
```

### How Loadable works

- load(request) increments a request counter, sets loading = true, clears the error and notifies listeners.
- When the response arrives, it is ignored if a newer request has already been sent. This stops an old search result from overwriting a newer one.
- On success it stores data, source and updatedAt; on failure it stores the error. The previous data is **kept**, so the screen can show the old data with an error banner instead of going blank.
- Every screen uses the same rule: no data + loading → skeleton; no data + error → full-screen ErrorView; data + error → RefreshErrorBanner over the old data; data + empty list → EmptyView.

### Watchlist state (optimistic updates)

- `toggle(id)` updates the local set and calls `notifyListeners()` **right away**, so the star fills and the watchlist row disappears instantly, before the server replies.
- It then calls POST or DELETE. On success it replaces the set with the server's list; on failure it rolls back and returns an error message, which the caller shows in a `SnackBar`.
- Star buttons use `context.select`, so only the star that changed rebuilds.

## 4.3 Networking and error handling

| Situation                                | What <code>ApiService</code> throws                                    | What the user sees                                                                                    |
|------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Backend not running / no network         | Could not reach the server at <url>. Make sure the backend is running. | <code>ErrorView</code> ('Connection lost') with <code>Retry</code> , or an error banner over old data |
| No response within 15 s                  | The server took too long to respond.                                   | Same as above                                                                                         |
| Invalid JSON / unexpected response shape | The server sent an invalid / unexpected response.                      | Same as above                                                                                         |
| HTTP 4xx/5xx with {error}                | The server's own error message                                         | e.g. 'Coin not found'                                                                                 |
| Watchlist toggle fails                   | Could not update watchlist: ...                                        | The star switches back and a <code>SnackBar</code> appears                                            |

## 4.4 When the app calls the backend

| Where in the app         | Endpoint                                                       | When it is called                                                                                                                                                       |
|--------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| App start                | GET <code>/api/watchlist/ids</code>                            | Once, so the right stars are filled                                                                                                                                     |
| Markets                  | GET <code>/api/coins?search&amp;filter;&amp;sort;</code>       | On open; 350 ms after typing stops (debounce); on filter or sort change; on pull-to-refresh; every 30 s (paused when the app is in the background or the tab is hidden) |
| Coin details             | GET <code>/api/coins/{id}</code> and <code>/chart?days=</code> | On open; on range change; on pull-to-refresh                                                                                                                            |
| Stats                    | GET <code>/api/global</code>                                   | On open; on pull-to-refresh; every 60 s                                                                                                                                 |
| Watchlist                | GET <code>/api/watchlist</code>                                | On open; when a coin is added; on pull-to-refresh; every 30 s                                                                                                           |
| Star button (any screen) | POST / DELETE <code>/api/watchlist/{id}</code>                 | On tap (optimistic)                                                                                                                                                     |

**NOTE** Even with many copies of the app refreshing every 30 s, the backend's 60-second cache means `CoinGecko` sees at most about one coin-list call per minute.

## SECTION 05

# Design System & Motion

Dark-only, crypto-style look: a deep-space background, cyan→violet neon highlights and frosted-glass panels. All design tokens are in `theme/app_theme.dart` and every screen reuses the same widgets.

## 5.1 Colour tokens (KxColors)

|                                                                                                                             |                                                                                                                               |                                                                                                                                 |                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|  <b>bg</b><br>#05060B<br>App background    |  <b>bgElevated</b><br>#0C0F18<br>Sheets, nav |  <b>cyan</b><br>#00E5FF<br>Primary accent      |  <b>violet</b><br>#8B5CF6<br>Secondary accent |
|  <b>magenta</b><br>#E040FB<br>Ambient glow |  <b>up</b><br>#00F5A0<br>Price up / gains    |  <b>down</b><br>#FF4D6D<br>Price down / errors |  <b>warn</b><br>#FFB547<br>Cached / star      |
|  <b>text</b><br>#EAF0FF<br>Primary text    |  <b>textDim</b><br>#9AA3B8<br>Secondary text |  <b>textMuted</b><br>#5F6780<br>Hints, ranks   |  <b>surface</b><br>#FFFFFF @ 8%<br>Glass fill |

Brand gradient: cyan #00E5FF → violet #8B5CF6 (top-left to bottom-right). It is used for primary buttons, the selected chip or segment, the nav bar highlight, gradient text and section markers.

## 5.2 Typography (google\_fonts)

| Role                            | Font                                                          | Used for                                                                                              |
|---------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Display (KxText.display)</b> | Space Grotesk, weight 600-700, slight negative letter spacing | Screen titles, coin symbols, headings, the wordmark                                                   |
| <b>Body (KxText.body)</b>       | Inter                                                         | Descriptions, labels, buttons, search                                                                 |
| <b>Numbers (KxText.mono)</b>    | JetBrains Mono with tabular figures                           | Every price, percentage, axis label and count, so digits line up and don't jump around as they change |
| <b>Label (KxText.label)</b>     | Inter semibold, wide letter spacing, uppercase                | Section headers (OVERVIEW, PORTFOLIO), badges                                                         |

## 5.3 Component library (lib/widgets)

| Component                           | What it is                                                                                                                                                                                   |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>KxBackground</b>                 | Animated backdrop painted with CustomPainter: three soft glows drifting on a 24 s loop plus a faint grid that scrolls slowly and fades out towards the bottom; wrapped in a RepaintBoundary. |
| <b>GlassCard</b>                    | Translucent panel with a hairline gradient border (custom painter), optional outer glow, optional blur (used only for floating elements), and an InkWell ripple.                             |
| <b>GradientText / SectionHeader</b> | Text filled with the brand gradient; uppercase section label with a small gradient marker.                                                                                                   |
| <b>AnimatedPrice</b>                | Rolls smoothly to a new price over 700 ms and flashes a green/red background when it changes; the flash can be turned off (used while scrubbing the chart).                                  |

| Component                                      | What it is                                                                                                                                                               |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ChangePill</b>                              | Green or red pill with an up/down arrow and the % change in mono.                                                                                                        |
| <b>CoinAvatar</b>                              | Coin logo (cached_network_image) in a thin ring; shows a gradient letter if the image fails; can act as a Hero so it flies between screens.                              |
| <b>WatchlistStar</b>                           | Star toggle with an elastic pop, shimmer and haptic feedback; rebuilds only itself.                                                                                      |
| <b>SourceBadge</b>                             | LIVE / CACHED / DEMO pill with a pulsing glowing dot and a tooltip.                                                                                                      |
| <b>Sparkline</b>                               | Custom-painted 7-day line that draws itself in over 900 ms, with a glow and gradient fill.                                                                               |
| <b>PriceChart</b>                              | fl_chart line chart: curved neon line with a glow, gradient fill, dashed grid, mono axis labels, crosshair and tooltip, scrub callback, and a left-to-right reveal clip. |
| <b>CoinTile / TrendingCard / TickerTape</b>    | Market row; carousel card; scrolling price ticker (two copies side by side so it loops seamlessly, width measured only when its content changes).                        |
| <b>KxSearchField / KxChipBar / KxSegmented</b> | Glass search box with a glowing focus ring; chips where the selected one has a gradient fill; segmented control whose gradient highlight slides between options.         |
| <b>KxNavBar</b>                                | Floating blurred-glass bottom bar; the glowing highlight slides between tabs with a slight bounce, and the active icon gets the gradient and grows slightly.             |
| <b>LoadingView / ErrorView / EmptyView</b>     | Shimmer skeleton rows; a pulsing error icon with Retry; a floating empty-state icon with an action button.                                                               |
| <b>GradientButton / RefreshErrorBanner</b>     | Main call-to-action button; a banner that slides in when a refresh fails while data is on screen.                                                                        |

## 5.4 Motion catalogue

| Animation                                     | How it's built                                                                                                                          | Timing           |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Splash intro</b>                           | Custom painter (ring sweep, hexagon, 'X' drawn stroke by stroke) + flutter_animate letter reveal, then a FadeThrough transition         | 1.8 s            |
| <b>Ambient background</b>                     | AnimationController.repeat + CustomPainter                                                                                              | 24 s loop        |
| <b>List entrance</b>                          | Rows fade and slide in one after another (40 ms apart, only the first 12 are delayed); replays only when search, filter or sort changes | about 400 ms     |
| <b>Price tick</b>                             | TweenAnimationBuilder rolls the number + a colour flash that fades out                                                                  | 700 / 900 ms     |
| <b>Chart</b>                                  | Reveal clip draws the line from left to right; fl_chart animates between data sets                                                      | 1.1 s / 350 ms   |
| <b>Sparkline</b>                              | Painter draws a growing share of the points                                                                                             | 900 ms           |
| <b>Hero logo</b>                              | Hero with a per-screen tag prefix (mkt, trend, gain, lose, vol, watch)                                                                  | Route transition |
| <b>Page transitions</b>                       | SharedAxis horizontal (animations package) on Android, Windows and Linux; Cupertino on iOS and macOS                                    | About 300 ms     |
| <b>Tab switch</b>                             | Fade and slight lift over the IndexedStack; hidden tabs have TickerMode off so their animations stop                                    | 320 ms           |
| <b>Nav / segmented highlight</b>              | AnimatedPositioned with easeOutBack                                                                                                     | 320-420 ms       |
| <b>Skeleton shimmer</b>                       | flutter_animate .shimmer, each row slightly delayed                                                                                     | 1.4 s loop       |
| <b>Count-up numbers</b>                       | TweenAnimationBuilder from 0 (stats tiles, watchlist average)                                                                           | about 1 s        |
| <b>Donut / gauge / range bar / supply bar</b> | Tweens from 0 to the value; donut sweeps round, gauge needle sweeps, bars fill                                                          | About 1 s        |

| Animation  | How it's built                                                                               | Timing     |
|------------|----------------------------------------------------------------------------------------------|------------|
| Live badge | Repeating fade on the dot                                                                    | 0.9 s loop |
| Haptics    | HapticFeedback on tab, chip, segment, star, chart scrub, swipe-to-remove and pull-to-refresh | -          |

## SECTION 06

## Screens: UI and Logic

The mock-ups below show each screen's layout, drawn in the app's own colours. Next to each one is what the screen shows, how its logic works, and how it handles loading, errors and empty results.

### 6.1 Splash screen (splash\_screen.dart)



**Purpose:** a branded 1.8-second intro that plays while the watchlist ids load in the background.

#### Sequence

- A gradient ring sweeps around with a glowing head while a rounded hexagon scales and rotates in with a pulsing cyan glow.
- The 'X' draws itself stroke by stroke inside the hexagon.
- 'Krypto' rises in letter by letter with a blur-in; the 'X' pops in as gradient text, then a shimmer runs across.
- The tagline CRYPTO MARKET INTELLIGENCE appears, with a gradient progress line and a SYNCING MARKETS nn% label.
- Then it replaces itself with HomeShell using a FadeThrough transition (protected against navigating twice).

#### Logic

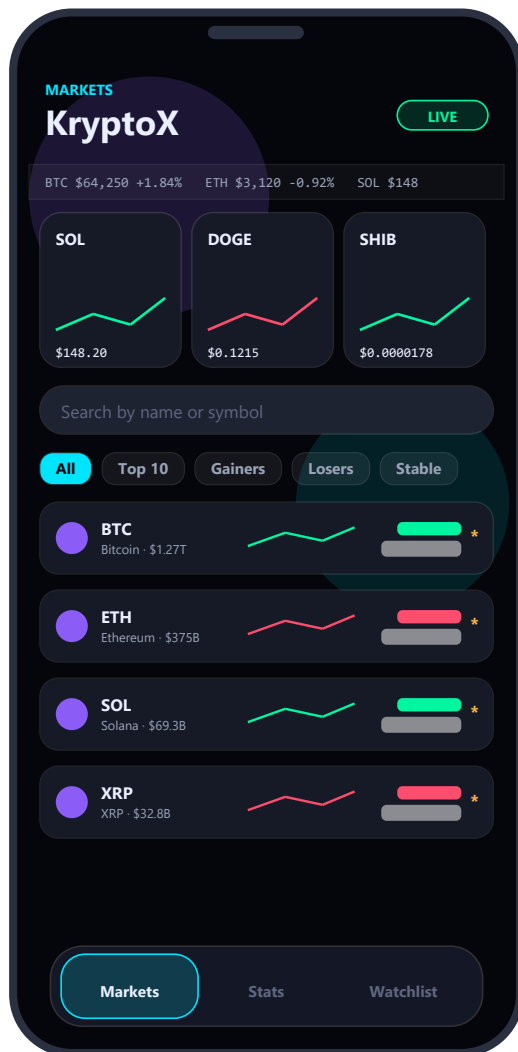
- Two AnimationControllers (TickerProviderStateMixin), both disposed.
- No network call of its own: the WatchlistProvider has already started loading from main().

Splash: logo ring sweep, wordmark reveal, progress line

### 6.2 App shell and navigation (home\_shell.dart)

- KxBackground → transparent Scaffold with `extendBody: true`, so content scrolls behind the floating glass KxNavBar.
- An IndexedStack keeps all three tabs alive (scroll position, search text, filters). Each tab is wrapped in `TickerMode(enabled: index == i)`, so hidden tabs stop animating and skip auto-refresh, which saves battery.
- Switching tabs runs a 320 ms fade and slight lift. Pressing back on Stats or Watchlist returns to Markets instead of leaving the app (PopScope).
- Tabs add 110 px of bottom padding (`kNavBarClearance`) so the last items aren't hidden behind the floating bar.
- Coin details are always opened with `openCoin(context, coin, heroPrefix: ...)`. Each list uses its own prefix so no two logos share a Hero tag.

## 6.3 Markets screen (coin\_list\_screen.dart)



Markets: header, ticker, trending, pinned search and filters, coin rows

### Layout (CustomScrollView, top to bottom)

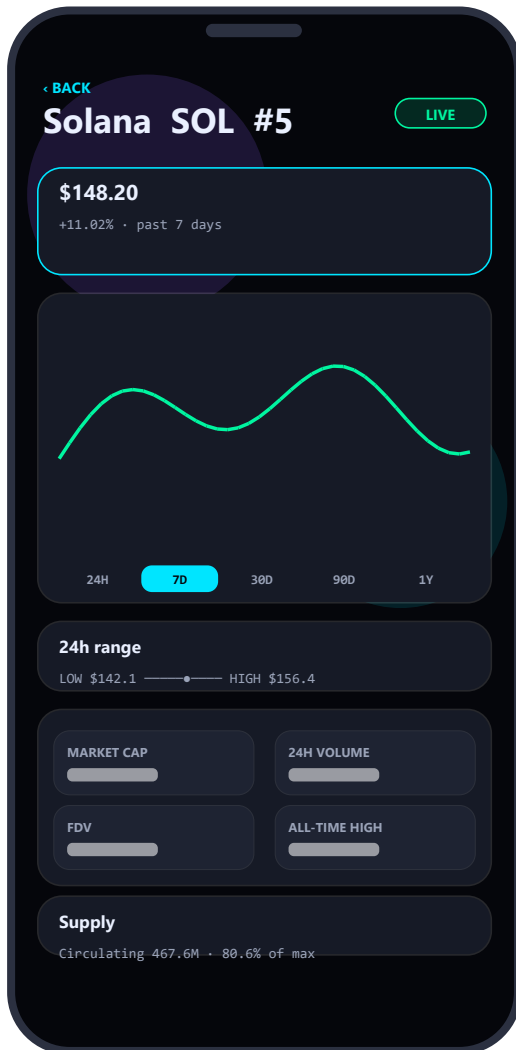
- Header: MARKETS label, gradient 'KryptoX' title, LIVE badge, sort button (with a dot when the sort isn't the default), 'Updated Xs ago' counting live, and a tappable 'Sorted by ...' line.
- TickerTape: the top 15 coins by rank scrolling endlessly.
- Trending: a carousel of the 5 coins with the biggest 24h move (up or down). It collapses when a search or filter is active.
- Search box + filter chips pinned while scrolling; they frost over the content once it scrolls underneath, with a thin loading line.
- Column headings (# / Asset / 7d / Price), then CoinTile rows animating in one after another.

### Logic

- Search: 350 ms debounce, then the search runs on the server.
- Filters and sort go to the server; the sort sheet stays open so you can change several options at once.
- Auto-refresh every 30 s: skipped while a load is running, while typing, when the tab is hidden, or when another page is on top; paused in the background and refreshed on return if the data is older than 30 s.
- The ticker and trending carousel always use the latest unfiltered result, so they stay visible during a search.

| State                             | What is shown                                                                                                 |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------|
| First load                        | LoadingView: shimmering skeleton rows shaped like coin rows                                                   |
| First load fails                  | ErrorView: pulsing icon, 'Connection lost', the error message, gradient Retry button                          |
| Refresh fails with data on screen | RefreshErrorBanner slides in above the nav bar; the old data stays visible                                    |
| No results                        | EmptyView: 'No coins found for "xyz"' or 'No coins match this filter', with a 'Clear search & filters' button |

## 6.4 Coin details screen (coin\_detail\_screen.dart)



Coin details: price that follows your finger on the chart, range bar, stats

**Opened with:** `CoinDetailScreen(coinId, initial: coin, heroTag)`. The row data passed in is shown immediately while the full details load, and the logo flies in from the list.

### Sections

- Header: glass back button, logo (Hero), name, symbol chip, rank chip, source badge, watchlist star.
- Price block: 36 px `AnimatedPrice` with a glow, and the % change for the selected range (from the chart's first and last points; before the chart loads it uses the API's 24h/7d/30d/1y figure).
- Chart card: a 260 px `PriceChart` with a `KxSegmented` control for 24H / 7D / 30D / 90D / 1Y.
- 24h range bar with a marker that slides to the current price; 4 performance tiles (24h, 7d, 30d, 1y).
- Market stats grid (10 tiles): market cap, volume, fully diluted valuation, volume/market-cap ratio, all-time high and date, % from ATH, all-time low and date, % from ATL.
- Supply card: circulating, total and max, with an animated progress bar. About: description with Read more, category pills, homepage.

### Chart scrubbing logic

- `PriceChart` reports the point under the finger through `onScrub`. The screen stores it in a `ValueNotifier`, so **only the price block** rebuilds while you drag, keeping it smooth.
- While dragging, the big price shows the historical price (without the flash), the pill shows the change from the start of the range, the caption shows that point's date and time, and a `HISTORICAL` tag appears. On release it rolls back to the live price.
- Each chart response remembers which range it was loaded for, so a slow old response can't be drawn under a newer range selection.

### States

- No coin data at all: a full-page loader, or `ErrorView` if it fails. If details fail but the list data exists: an inline error card with `Retry` while the header and chart keep working.
- Chart: a shimmer placeholder on first load; a dimmed 'Loading 30D' overlay when changing range; an inline error with `Retry` if a range fails; a small 'Update failed · `Retry`' chip if a refresh fails; an empty state if there are fewer than 2 points.

## 6.5 Market stats screen (market\_stats\_screen.dart)



Stats: hero card, KPIs, dominance donut, sentiment gauge, top movers

### Sections

- Header: OVERVIEW label, gradient 'Global Market' title, source badge, time of last update.
- Hero card: total market cap (AnimatedPrice, compact), 24h change pill, and a '7D PULSE' sparkline: an index of the top-volume coins' 7-day sparklines, weighted by market cap.
- KPI grid (6 tiles, numbers count up): 24h volume, volume/market-cap ratio, BTC dominance, ETH dominance, active coins, markets.
- Dominance donut (fl\_chart PieChart): top 5 + Others in each coin's brand colour. It sweeps in, and tapping a slice or legend row highlights it.
- Sentiment gauge (CustomPainter): a semicircle with a sweeping needle, labelled ESTIMATE.
- Top movers: a Gainers / Losers / Volume switch that cross-fades between lists.

### Logic

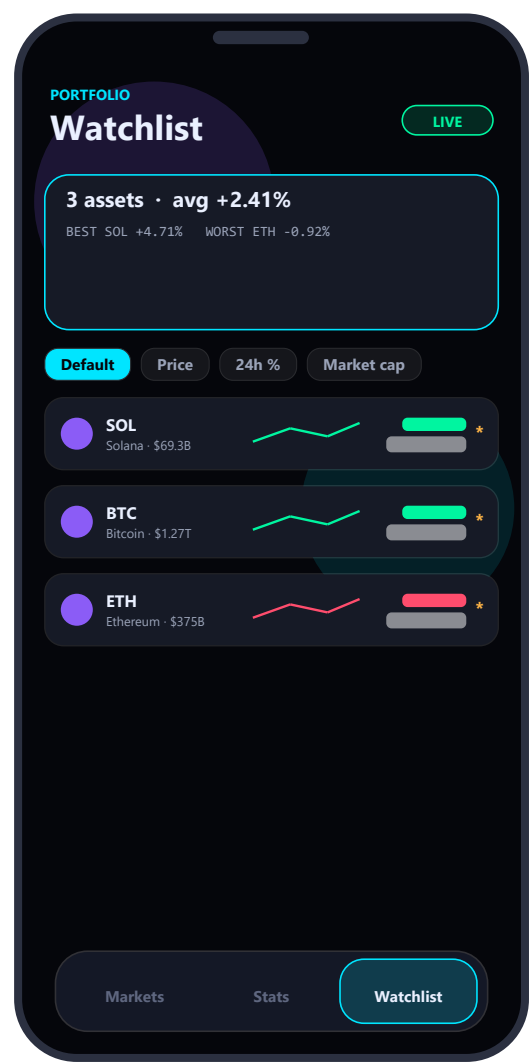
- Auto-refresh every 60 s; pull-to-refresh; sections animate in only on the first load; RefreshErrorBanner if a refresh fails.

### Sentiment estimate formula (calculated in the app, not an official index)

```
momentum = clamp(marketCapChange24h × 6, -30, +30)           // how far the whole market moved today
g = average 24h change of top gainers, l = average of top losers
balance  = (g + 1) / (|g| + |l|) × 20                        // -20 ... +20: did gainers outweigh losers?
score    = clamp(50 + momentum + balance, 0, 100)

label: < 25 Extreme Fear · < 45 Fear · < 55 Neutral · < 75 Greed · otherwise Extreme Greed
```

## 6.6 Watchlist screen (watchlist\_screen.dart)



Watchlist: summary card, sort chips, swipe-to-remove rows

### Sections

- Header: PORTFOLIO label, gradient 'Watchlist' title, source badge.
- Summary card: number of assets, average 24h change (counts up, coloured), a custom-painted strip of each coin's 24h change from best to worst, and BEST / WORST chips with logos.
- Sort chips (sorted in the app): Default (order added), Price, 24h %, Market cap; coins with no value go last.
- CoinTile rows (heroPrefix 'watch'), each animating in only the first time it appears.

### Logic

- Listens to WatchlistProvider. **Removals** are filtered out locally straight away; **additions** (a coin starred on another tab) trigger a reload to fetch that coin's data.
- Swipe left to remove: a red gradient background with a trash badge that grows as you drag, a haptic tick at the threshold, then a SnackBar with **Undo**. The coin is removed from the list in the same frame, as Flutter's Dismissible requires.
- Auto-refresh every 30 s; pull-to-refresh.

| State   | What is shown                                                                                                              |
|---------|----------------------------------------------------------------------------------------------------------------------------|
| Loading | LoadingView with 4 skeleton rows                                                                                           |
| Error   | ErrorView with Retry                                                                                                       |
| Empty   | EmptyView: floating star icon, 'Your watchlist is empty', and an 'Explore markets' button that switches to the Markets tab |

## SECTION 07

# Performance, Quality & Robustness

## 7.1 Performance decisions

- Blur (BackdropFilter) is used only on floating elements (nav bar, search box, pinned filter bar) and never on individual list rows.
- Repaint boundaries around the animated background and the ticker, so their per-frame repaints don't force the rest of the screen to repaint.
- Hidden tabs have TickerMode turned off, so their animations and auto-refresh stop.
- The ticker measures its width only when its content changes, not on every frame.
- context.select for watchlist stars, a ValueNotifier for chart scrubbing, and ListenableBuilder, so only the widgets that change rebuild.
- Entrance animations are limited (only the first 12 rows are delayed) and don't replay on the 30-second refresh.
- Coin logos are cached on disk with cached\_network\_image.

## 7.2 Robustness

- Every Timer, AnimationController, ValueNotifier, Loadable and listener is disposed; every setState after an await first checks mounted.
- JSON parsing tolerates null items in lists, ints where decimals are expected, and missing fields. A response that isn't a JSON object becomes a readable error.
- Charts guard against a zero or flat price range (e.g. stablecoins) and against an out-of-range touch index while switching ranges.
- Unique Hero tag prefixes, because the three tabs are all in the widget tree at the same time.

## 7.3 Unit tests (test/formatters\_test.dart)

| Test                 | What it checks                                                                                              |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| <b>formatPrice</b>   | \$64,250.12 for large prices; 4 to 8 decimals for small prices (\$0.5850, \$0.00001780); '—' for null       |
| <b>formatCompact</b> | \$1.27T, \$28.40B, 19.74M without the \$ sign, \$950.00                                                     |
| <b>formatPercent</b> | +1.84% / -0.90%                                                                                             |
| <b>Coin.fromJson</b> | Converts ints to decimals, upper-cases the symbol, tolerates nulls and missing fields, parses the sparkline |

## SECTION 08

# Setup, Run & Test Guide

## 8.1 Requirements

- Flutter SDK 3.38 or newer (which includes Dart). Check with **flutter --version**.
- An Android emulator or device, Chrome, or Windows desktop.
- Internet access for live data; without it, use demo mode (USE MOCK=1).

## 8.2 Steps

```
# Terminal 1: backend
cd backend
dart pub get
dart run bin/server.dart           # http://localhost:8080

# Terminal 2: app
cd flutter_app
.\setup.ps1                        # once: flutter create for android/ios/web/windows,
                                  # adds the INTERNET permission, allows plain HTTP, sets the app label "KryptoX"

flutter run                        # choose an emulator, Chrome or Windows
flutter test                       # unit tests

# Real phone on the same Wi-Fi:
flutter run --dart-define=API_BASE_URL=http://<your-PC-IP>:8080
```

## 8.3 Testing the backend by hand

```
curl http://localhost:8080/api/health
curl "http://localhost:8080/api/coins?filter=gainers&sort=change_24h"
curl http://localhost:8080/api/coins/bitcoin
curl "http://localhost:8080/api/coins/bitcoin/chart?days=30"
curl http://localhost:8080/api/global
curl -X POST http://localhost:8080/api/watchlist/solana
curl http://localhost:8080/api/watchlist
curl -X DELETE http://localhost:8080/api/watchlist/solana
```

## 8.4 Troubleshooting

| Symptom                                 | Cause / fix                                                                                                                                                                                   |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'Could not reach the server' in the app | The backend isn't running, or the address is wrong. The emulator uses 10.0.2.2; a real phone needs --dart-define=API_BASE_URL with the PC's IP address and the firewall must allow port 8080. |
| Android blocks HTTP                     | Run setup.ps1 (it adds android:usesCleartextTraffic="true" and the INTERNET permission).                                                                                                      |
| The badge shows DEMO or CACHED          | CoinGecko is rate-limiting or unreachable. Wait a minute, or set COINGECKO_API_KEY.                                                                                                           |
| Package version conflicts               | Use Flutter 3.38 or newer. animations and cached_network_image are pinned to their last versions built on Flutter's standard material.dart (see 9.2).                                         |

## SECTION 09

# Verification Status & Roadmap

## 9.1 Verification status

| Part        | Status                                 | Detail                                                                                                                                                                                                                                                                                                                                                                          |
|-------------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend     | Tested                                 | dart analyze: no issues. Every endpoint checked against live CoinGecko and in demo mode, including the 400 and 404 errors.                                                                                                                                                                                                                                                      |
| Flutter app | Written and reviewed, not yet compiled | Built by several agents working in parallel, then reviewed line by line by two more. Every third-party call was checked against the real package sources (fl_chart 1.2, flutter_animate 4.5, google_fonts 8.2, animations 2.2, cached_network_image 3.4); 14 issues were fixed. The first 'flutter run' happens on the target machine; small compile fixes may still be needed. |
| Unit tests  | Written                                | Run with flutter test on the target machine.                                                                                                                                                                                                                                                                                                                                    |

## 9.2 Dependency note

Flutter is in the middle of moving Material widgets out of the SDK into a separate **material\_ui** package. animations 3.x and cached\_network\_image 4.x already use material\_ui, while fl\_chart and google\_fonts still use the SDK's package:flutter/material.dart. Mixing the two would cause type conflicts (ThemeData and PageTransitionsBuilder would be different classes). KryptoX therefore stays on flutter/material and pins animations ^2.2.0 and cached\_network\_image ^3.4.1, which have the same API.

## 9.3 Limitations and production roadmap

| Current limitation                                                  | Production step                                                                                                   |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| One shared watchlist; no user accounts                              | Add authentication (JWT / OAuth) and store each user's watchlist in a database (Postgres or SQLite)               |
| The cache lives in memory, is lost on restart and has no size limit | Redis, or an in-memory cache with a size limit and eviction; use CoinGecko's own freshness hints                  |
| CORS allows every website (*); plain HTTP                           | Allow only your own domain; put it behind HTTPS with a reverse proxy (Caddy or nginx) or a cloud load balancer    |
| Top 100 coins, USD only                                             | Add paging (page and per_page) and a vs_currency parameter                                                        |
| Prices refresh by polling every 30 s                                | Push updates to the app over WebSocket or server-sent events                                                      |
| The sentiment gauge is an estimate                                  | Use a published fear & greed index as its source                                                                  |
| No CI or automated deployment                                       | GitHub Actions running flutter analyze, flutter test and the backend tests; build the backend into a Docker image |

## SECTION 10

# Appendix: Source File Inventory

| File                                 | Lines   | Contents                                                              |
|--------------------------------------|---------|-----------------------------------------------------------------------|
| backend/bin/server.dart              | 235     | Pipeline, routes, search, filter, sort, global stats, watchlist, CORS |
| backend/lib/coingecko.dart           | 107     | HTTP client, TTL cache with stale fallback, normalisers               |
| backend/lib/mock_data.dart           | 153     | 20 demo coins, repeatable random-walk charts, global stats            |
| lib/main.dart                        | 54      | Providers, theme, system UI, splash                                   |
| lib/config.dart                      | 15      | Base URL resolution                                                   |
| lib/models/coin.dart                 | 163     | Data models and JSON parsing                                          |
| lib/services/api_service.dart        | 175     | REST client, ApiException, CoinSort / CoinFilter                      |
| lib/state/loadable.dart              | 49      | Loading, error and data holder                                        |
| lib/state/watchlist_provider.dart    | 44      | Optimistic watchlist state                                            |
| lib/theme/app_theme.dart             | 124     | KxColors, KxText, ThemeData, transitions                              |
| lib/widgets/kx_background.dart       | 97      | Animated backdrop                                                     |
| lib/widgets/glass.dart               | 153     | GlassCard, GradientText, SectionHeader                                |
| lib/widgets/market_widgets.dart      | 246     | AnimatedPrice, ChangePill, CoinAvatar, WatchlistStar, SourceBadge     |
| lib/widgets/coin_tile.dart           | 248     | CoinTile, TrendingCard, TickerTape                                    |
| lib/widgets/sparkline.dart           | 84      | Animated sparkline painter                                            |
| lib/widgets/price_chart.dart         | 198     | Interactive fl_chart line chart                                       |
| lib/widgets/controls.dart            | 325     | Search box, chip bar, segmented control, nav bar                      |
| lib/widgets/state_views.dart         | 223     | Loading, error and empty views, buttons, banner                       |
| lib/widgets/formatters.dart          | 73      | Price, number, percent and date formatting                            |
| lib/screens/splash_screen.dart       | 371     | Animated intro                                                        |
| lib/screens/home_shell.dart          | 84      | Tabs, nav bar, back handling                                          |
| lib/screens/navigation.dart          | 12      | openCoin() helper                                                     |
| lib/screens/coin_list_screen.dart    | 900     | Markets                                                               |
| lib/screens/coin_detail_screen.dart  | 1,365   | Coin details                                                          |
| lib/screens/market_stats_screen.dart | 1,085   | Global stats dashboard                                                |
| lib/screens/watchlist_screen.dart    | 673     | Watchlist                                                             |
| Total                                | ≈ 7,256 |                                                                       |